

## Epic 02 Completion Criteria

### 1. What is a Spring Boot microservice?

A microservice is a **small, independent application that handles one business responsibility**.

Example, in a ride-booking system:

- **User Service** → manages users.
- **Driver Service** → manages drivers.
- **Ride Service** → manages rides.
- **Payment Service** → manages payments.

Each service can run independently and communicate with other services through APIs.

### 2. How to create a microservice from scratch

We should know the complete process:

**Step 1:** Create a Spring Boot project.

**Step 2:** Add required dependencies.

**Step 3:** Create the package structure.

**Step 4:** Create Entity, Repository, Service, and Controller.

**Step 5:** Configure the database.

**Step 6:** Create APIs.

**Step 7:** Test the APIs using Postman.

**Step 8:** Run and demonstrate the application.

Example:      User Microservice

v

Controller

v

Service

v

Repository

v

Database

### 3. Controller → Service → Repository architecture

#### Controller

Receives the HTTP request.

Example:

```
@GetMapping("/{id}")  
  
public UserResponse getUser(@PathVariable Long id) {  
    return userService.getUser(id);  
}
```

#### Service

Contains the business logic.

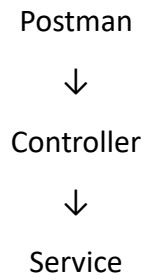
```
public UserResponse getUser(Long id) {  
    User user = userRepository.findById(id)  
        .orElseThrow(() -> new RuntimeException("User not found"));  
    return new UserResponse(user.getId(), user.getName());  
}
```

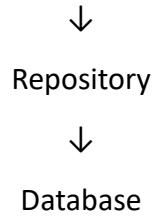
#### Repository

Communicates with the database.

```
public interface UserRepository  
    extends JpaRepository<User, Long> {  
}
```

#### Flow:





#### 4. Entity vs DTO

##### Entity

Represents the **database table**.

@Entity

```
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String name;  
    private String email;  
}
```

##### DTO

Represents the **data sent or received through the API**.

```
public class UserResponse {  
    private Long id;  
    private String name;  
    public UserResponse(Long id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

## Main difference

| Entity                            | DTO                         |
|-----------------------------------|-----------------------------|
| Used for database                 | Used for API                |
| Represents table                  | Represents request/response |
| Contains database-related details | Contains only required data |

### Example:

Database has id, name, email, password.

API response should return only:

```
{  
  "id": 1,  
  "name": "Ravi"  
}
```

## 5. Request and Response DTOs

### Request DTO

Data coming **from the client**.

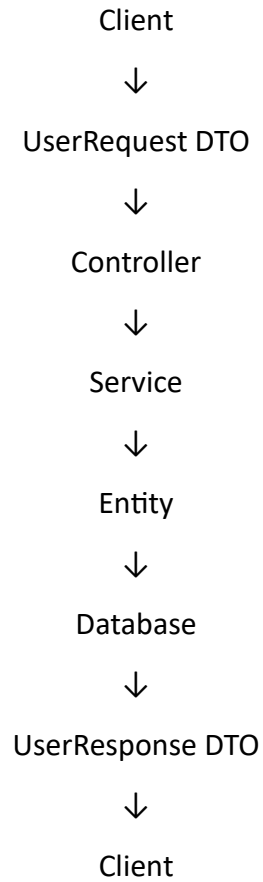
```
public class UserRequest {  
    private String name;  
    private String email;  
}
```

### Response DTO

Data going **to the client**.

```
public class UserResponse {  
    private Long id;  
    private String name;  
    private String email;  
}
```

**Flow:**



**6. Bean validation**

Validation checks whether the incoming data is correct.

Example:

```
public class UserRequest {  
    @NotBlank(message = "Name is required")  
    private String name;  
    @Email(message = "Invalid email")  
    @NotBlank(message = "Email is required")  
    private String email;  
}
```

Controller:

@PostMapping

```
public UserResponse createUser(  
    @Valid @RequestBody UserRequest request) {  
    return userService.createUser(request);  
}
```

If the client sends:

```
{  
    "name": "",  
    "email": "abc"  
}
```

The API should return a validation error instead of saving invalid data.

## 7. Global exception handling

Instead of writing try-catch in every controller, we use **@RestControllerAdvice**.

Example:

@RestControllerAdvice

```
public class GlobalExceptionHandler {  
    @ExceptionHandler(UserNotFoundException.class)  
    public ResponseEntity<String> handleUserNotFound(  
        UserNotFoundException ex) {  
        return ResponseEntity  
            .status(HttpStatus.NOT_FOUND)  
            .body(ex.getMessage());  
    }  
}
```

Now, if a user is not found:

```
{  
  "message": "User not found"  
}
```

with status:

404 NOT\_FOUND

## 8. HTTP status codes

You should know when to use each status code.

| Status                           | Meaning                      | Example              |
|----------------------------------|------------------------------|----------------------|
| <b>200 OK</b>                    | Request successful           | Get user             |
| <b>201 CREATED</b>               | New resource created         | Create user          |
| <b>204 NO CONTENT</b>            | Successful, no response body | Delete user          |
| <b>400 BAD REQUEST</b>           | Invalid request              | Validation failure   |
| <b>404 NOT FOUND</b>             | Resource not found           | User does not exist  |
| <b>409 CONFLICT</b>              | Duplicate/conflicting data   | Email already exists |
| <b>500 INTERNAL SERVER ERROR</b> | Unexpected server error      | Database failure     |

## 9. Spring Boot profiles

Profiles allow you to use **different configurations for different environments**.

Example:

application.properties

application-dev.properties

application-test.properties

application-prod.properties

**Development** :spring.datasource.url=jdbc:h2:mem:testdb

**Production** :spring.datasource.url=\${DB\_URL}

Activate a profile: spring.profiles.active=dev

## 10. Environment variables

Environment variables store configuration **outside the source code**.

Example:

```
spring.datasource.url=${DB_URL}
```

```
spring.datasource.username=${DB_USERNAME}
```

```
spring.datasource.password=${DB_PASSWORD}
```

Instead of writing the actual password in GitHub, you configure it in the environment.

## 11. Database-per-service

In microservices, each service should **own its own database**.

Example:

User Service → User Database

Ride Service → Ride Database

Payment Service → Payment Database

The Ride Service should not directly access the User Service's database.

If Ride Service needs user details, it should call the **User Service API**.